

# SafeMate — Phase 12.4.0 Audit Report

Conflict Resolution, Versioning & Server Reconciliation Architecture Audit

VERIFIED TESTS

381 Passing PASS

DART ANALYZER

0 Issues PASS

ANDROID BUILD

APK Debug (63.5s)

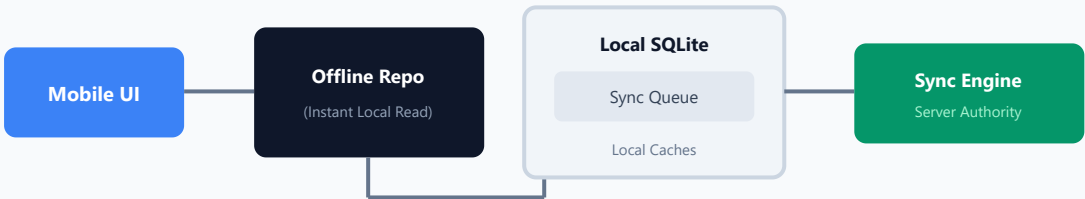
PASS

AUDIT STATUS

PAUSED FOR REVIEW

## 1. Executive Summary & Verification State

This document establishes the official technical audit for **Phase 12.4 (Conflict Resolution & Server Reconciliation)** of SafeMate. Following Phase 12.3, SafeMate operates with an offline-first foundation consisting of persistent SQLite caches, single-flight SyncEngine, network abstraction via ConnectivityService, and bounded retry mechanics. This audit analyzes every edge case across **data versioning, concurrent modification, delete/update races, wall-clock reliance, and server authority**.



## 2. Sync Queue Audit Findings

| Dimension       | Actual Implementation                                                              | Assessment                                         |
|-----------------|------------------------------------------------------------------------------------|----------------------------------------------------|
| Active Table    | sync_queue in SQLite (legacy fallback also defined as local_sync_queue in schema). | <span>MEDIUM</span> Harmonize table names.         |
| Operation ID    | UUIDv4 generated via Uuid().v4(); preserved across retries.                        | <span>OPTIMAL</span> Idempotency preserved.        |
| User Scoping    | Mandatory user_id column; checked in loop and purged on logout.                    | <span>SECURE</span> Multi-user isolation intact.   |
| Retry Policy    | Bounded at 5 attempts; backoff = min(2^retry, 60) + jitter(0..2) seconds.          | <span>OPTIMAL</span> No endless retry loops.       |
| Error Handling  | Classifies transient, permanent, conflict, authentication.                         | <span>OPTIMAL</span> 409 treated as non-retryable. |
| Causal Ordering | Flat FIFO (created_at ASC); no causal dependency graph between entities.           | <span>HIGH</span> Independent mutations may race.  |

### 3. Entity Versioning Audit Matrix

Every offline-syncable entity was audited for local and remote revision tracking:

| Entity           | Local Version     | Base Server Version | Server Version | Timestamp Field         | Audit Status & Risk                                        |
|------------------|-------------------|---------------------|----------------|-------------------------|------------------------------------------------------------|
| Trip             | None (0)          | None                | None           | updated_at (Wall-Clock) | <b>CRITICAL</b> Susceptible to clock skew / LWW overwrite. |
| Profile          | None              | None                | None           | updated_at              | <b>HIGH</b> Unconditional whole-record overwrite.          |
| Trip Preferences | None              | None                | None           | updated_at              | <b>HIGH</b> No conflict detection.                         |
| Itinerary        | None              | None                | None           | updated_at              | <b>CRITICAL</b> Document-level JSON overwrite.             |
| Chat Message     | client_message_id | None                | id (UUID)      | created_at              | <b>CONTROLLED</b> Server RPC deduplicates.                 |
| SafeTrip         | None              | None                | None           | updated_at              | <b>CONTROLLED</b> Server-authoritative RPCs only.          |
| Check-in         | idempotency_key   | None                | id (UUID)      | completed_at            | <b>OPTIMAL</b> Unique constraint on server.                |

### 4. Conflict Handling & Race Condition Analysis

**Key Finding — Unconditional LWW Detected:** In `OfflineFirstTripRepository._handleSyncMutation`, the system compares `remoteExisting.updatedAt.isAfter(trip.updatedAt)`. Because `trip.updatedAt` originates from the client device's clock, a device with a clock set ahead will **unconditionally overwrite newer server state**.

#### Conflict Scenario Breakdown:

- **A. Concurrent Update (Device A vs Device B):** **HIGH RISK**. No optimistic locking (version/etag) on Supabase update. The last HTTP request to reach the server wins.
- **B. Stale Offline Update:** **PARTIALLY HANDLED**. Trip repository detects divergence if server timestamp is ahead, marking operation as `conflict`. However, the conflicted record remains in SQLite without user resolution UI.
- **C. Delete vs Update:** **HIGH RISK**. If a trip draft is deleted on the server, an offline client that updates the draft will receive a 404. The sync engine marks it as failed, leaving local SQLite out of sync.
- **D. Realtime vs HTTP Race:** **MEDIUM RISK**. In chat, if Realtime delivers before HTTP returns, SQLite holds two rows temporarily because `local_chat_messages` lacks a unique index on `client_message_id`.

### 5. Server Authority Boundaries

| Domain Field                         | Authority   | Offline Capability    | Enforcement Status                                                                        |
|--------------------------------------|-------------|-----------------------|-------------------------------------------------------------------------------------------|
| <b>Verification Status</b>           | Server Only | Read Only (Preserved) | <b>ENFORCED</b> Local client resets to existing server flag.                              |
| <b>Trust Score &amp; Reliability</b> | Server Only | Read Only             | <b>HIGH GAP</b> Server RLS currently allows updating own profile without column triggers. |
| <b>Emergency / SOS</b>               | Server Only | Network Required      | <b>ENFORCED</b> Fails fast if offline; never queued silently.                             |
| <b>SafeTrip State Transition</b>     | Server Only | Network Required      | <b>ENFORCED</b> Activation/completion requires online server RPC.                         |

## 6. Complete Gap Analysis & Severity Ratings

| Area                  | Deficiency / Vulnerability                                                                 | Severity | Phase 12.4 Remediation Requirement                                                    |
|-----------------------|--------------------------------------------------------------------------------------------|----------|---------------------------------------------------------------------------------------|
| Versioning            | No monotonically increasing version / revision integer on trips, profiles, or itineraries. | CRITICAL | Add version INTEGER NOT NULL DEFAULT 1 and increment triggers on Supabase and SQLite. |
| Optimistic Locking    | Supabase update operations do not enforce WHERE version = base_version.                    | CRITICAL | Implement optimistic concurrency check (return 409 when version diverged).            |
| Clock Dependency      | Conflict detection in OfflineFirstTripRepository uses trip.updatedAt (device clock).       | CRITICAL | Replace timestamp comparison with base_version == server_version.                     |
| Chat Outbox Race      | local_chat_messages has primary key on id without unique index on client_message_id.       | HIGH     | Add UNIQUE(client_message_id) to SQLite chat schema and upsert by client ID.          |
| Itinerary Granularity | Itineraries are stored as single monolithic days_json blobs.                               | HIGH     | Implement field-level / day-level diffing or document-level version checks.           |
| Tombstone / Deletes   | No tombstone table or soft deletes for trips; deleted server rows cause 404 failures.      | MEDIUM   | Introduce local tombstone tracking or treat 404 on update as server-deleted conflict. |
| Conflict UI           | No user interface or controller notification when a record transitions to conflict.        | MEDIUM   | Provide conflict banner & resolution options (Keep Mine / Accept Server / View Diff). |

## 7. Recommended Phase 12.4 Implementation Roadmap

- Database Migration (Supabase & SQLite):** Add version columns to trips, profiles, and trip\_preferences with automatic increment triggers.
- Deterministic Optimistic Locking:** Pass base\_version in sync queue payloads; reject mutations on server when current version exceeds base version.
- Reconciliation Engine:** Replace wall-clock comparisons with version-based 3-way reconciliation (Server Wins for trust/safety; User Prompt for draft/profile content).
- Chat Outbox Idempotency:** Index client\_message\_id in SQLite to prevent dual-rendering during Realtime/HTTP races.
- Comprehensive Conflict & Chaos Test Suite:** Add 20+ automated tests covering concurrent edits, clock skew, network drop during commit, and conflict UI callbacks.

**Phase 12.4.0 Gate Status:** All code modifications remain paused. Ready for engineering review before proceeding to Phase 12.4.1 execution.